

1 Summary

2 Modern scientific research requires consistent document preparation, re-
3 producible computation, unified data management, and transparent figure
4 provenance—yet researchers rely on fragmented tools that increase cognitive
5 load and introduce irreproducibility. We present SciTeX, an AI-native, mod-
6 ular research automation platform providing four integrated modules: Writer
7 for structured LaTeX manuscript preparation with containerized builds; Scholar
8 for literature management with automated metadata extraction; Viz for
9 publication-quality figure generation with embedded provenance; and Code
10 for reproducible computation with GPU and container support. Each mod-
11 ule operates independently while sharing a unified project structure through
12 lightweight symlinks and global identifiers, enabling cross-module traceabil-
13 ity. SciTeX reduces the activation energy required for rigorous, reproducible
14 science while maintaining compatibility with traditional workflows. The plat-
15 form is open source and follows FAIR principles. This manuscript demon-
16 strates SciTeX’s self-descriptive capabilities by being prepared within the
17 platform it documents.

18 Highlights

- 19 • SciTeX integrates manuscript writing, figure generation, literature man-
20 agement, and computation in a unified platform
- 21 • Modular architecture enables independent use of each component while
22 providing synergy through integration
- 23 • Containerized compilation ensures reproducible manuscript output across
24 all operating systems
- 25 • Embedded metadata enables full provenance tracking from figures and
26 results back to generating code
- 27 • AI-native design provides integration points for LLM-assisted research
28 workflows

SciTeX: An AI-Native Modular Platform for Reproducible Scientific Research

Yusuke Watanabe^{a,*}

^a*SciTeX.ai, Tokyo, Japan*

Keywords: reproducible research, scientific workflow automation, data science platform, AI-native tools, figure provenance, FAIR principles

~ 8 figures, 3 tables, 136 words for abstract, and 2598 words for main text

1. Introduction

Scientific workflows increasingly span multiple heterogeneous environments: personal laptops, HPC systems, cloud servers, and containerized pipelines. Traditional manuscript preparation tools remain fragmented, requiring manual management of figures, references, code execution, and version control. These inconsistencies lead to irreproducible results, duplicated effort, and high cognitive load that distracts researchers from their primary scientific objectives.

The preparation of scientific manuscripts involves numerous technical challenges that extend beyond the intellectual task of communicating research findings. Researchers must navigate complex typesetting systems, manage multiple document versions, coordinate figures and tables across formats, and ensure reproducible compilation environments. Furthermore, maintaining consistency between computational analyses, their visual representations, and the manuscript text requires constant manual synchronization—a process prone to error and version drift.

Traditional approaches to manuscript preparation typically rely on local LaTeX installations, where specific versions of packages and compilation

*Corresponding author. Email: y.watanabe@scitex.ai

55 tools can vary significantly across machines and over time. This variability
56 creates reproducibility challenges, particularly in collaborative environments
57 where multiple authors work on different systems. Similarly, figure genera-
58 tion often occurs in isolation from the manuscript, with researchers using var-
59 ious plotting libraries without standardized metadata or provenance tracking.
60 Literature management frequently depends on external tools that operate in-
61 dependently from the writing environment, leading to citation inconsistencies
62 and manual bibliography maintenance.

63 Existing solutions have addressed individual aspects of this problem.
64 Overleaf and similar cloud-based platforms provide consistent compilation
65 environments but require continuous internet connectivity and separate fig-
66 ure generation workflows. Zotero and Mendeley excel at citation management
67 but lack integration with manuscript preparation and computational anal-
68 ysis. Jupyter notebooks enable reproducible computation but provide lim-
69 ited support for publication-quality figures or structured manuscript writing.
70 SigmaPlot and Origin offer sophisticated visualization capabilities but op-
71 erate as standalone applications without provenance tracking or manuscript
72 integration.

73 The fundamental challenge lies in unifying these disparate tools while
74 maintaining the flexibility that researchers require. The solution must accom-
75 modate diverse content types, multiple output documents, and varying jour-
76 nal requirements while ensuring a single source of truth for shared elements.
77 Additionally, modern research increasingly involves AI-assisted workflows for
78 code generation, literature review, and manuscript revision—capabilities that
79 require native integration rather than ad-hoc addition.

80 SciTeX addresses these challenges through a unified, modular, AI-native
81 platform that allows researchers to: write manuscripts in a structured LaTeX
82 environment with containerized compilation; manage citations and PDFs
83 with consistency and automated metadata extraction; generate publication-
84 quality figures with embedded provenance metadata; and execute analyses
85 in controlled environments with synchronized results. Importantly, SciTeX

86 maintains bidirectional low migration cost: each module functions indepen-
87 dently, providing value without requiring adoption of the entire ecosystem,
88 while synergy across modules enables a strictly superior integrated workflow.

89 Many researchers lack unified tools to integrate writing, computation, vi-
90 sualization, and literature management within a single, reproducible environ-
91 ment. Graduate students and early-career researchers often spend dispropor-
92 tionate time learning fragmented toolchains rather than conducting research.
93 Computational scientists require reproducible pipelines that connect analysis
94 code to manuscript figures without manual intervention. AI-augmented re-
95 search workflows demand native integration points for automated assistance.
96 SciTeX provides this missing infrastructure, reducing the activation energy
97 required for rigorous, reproducible science while democratizing access to ad-
98 vanced computational workflows for researchers unfamiliar with Linux, Git,
99 or containers.

100 This manuscript is prepared within SciTeX Writer, referencing figures
101 generated in SciTeX Viz, bibliography managed in SciTeX Scholar, and com-
102 putational environments controlled by SciTeX Code—thereby making the
103 manuscript a live demonstration of the platform’s integrated capabilities.

104 **2. Methods**

105 *2.1. System Architecture*

106 SciTeX consists of a Django-based cloud platform (SciTeX Cloud) and
107 corresponding Python packages installable via `pip install scitex`. The
108 architecture is modular, with each component operating independently while
109 sharing a unified project structure:

```
110 scitex/  
111     writer/    # Manuscript preparation  
112     scholar/   # Literature management  
113     viz/       # Figure generation  
114     code/      # Reproducible computation  
115     files/     # Shared assets
```

Each module is connected through lightweight symlinks, enabling seamless integration without tight coupling. Global identifiers ensure traceability between figures, code, bibliography entries, and manuscript content. All entities—figures, tables, code cells, literature entries—receive persistent IDs, enabling cross-module traceability, metadata embedding in PNG/JSON exports, and reverse lookup capabilities (e.g., “which script generated this figure?”).

2.2. *Writer Module*

The Writer module implements structured LaTeX-based manuscript preparation with the following capabilities:

Section-Separated Authoring: Manuscripts are organized into independent LaTeX files for each section (introduction, methods, results, discussion), facilitating modular development and enabling multiple authors to work simultaneously without merge conflicts.

Containerized Compilation: The framework leverages Docker and Apptainer/Singularity containerization to ensure reproducible builds. The compilation environment encapsulates specific versions of TeX Live and all required packages, eliminating dependency on host system configurations. This guarantees identical PDF output across macOS, Linux, Windows, and HPC environments.

Multi-Engine Support: Three compilation engines are supported with automatic selection: Tectonic for ultra-fast incremental builds (1–3 seconds), latexmk for reliable industry-standard compilation (3–6 seconds), and traditional 3-pass compilation for maximum compatibility (12–18 seconds).

Drag-and-Drop Integration: Figures generated in Viz and citations managed in Scholar can be inserted directly into manuscripts through drag-and-drop operations, with automatic metadata linking.

Version Tracking: Integration with Git provides systematic version tracking and automatic generation of difference documents using latexdiff, facilitating peer review processes.

146 2.3. Viz Module

147 The Viz module provides publication-quality figure generation with em-
148 bedded provenance metadata:

149 **SigmaPlot-Inspired Interface:** A graphical canvas interface enables
150 interactive figure construction while maintaining full programmatic control
151 through the matplotlib and seaborn APIs.

152 **Programmable Metadata:** Each figure exports with JSON sidecar
153 metadata encoding axis bounds, tick locations, panel geometry, color palettes,
154 statistical annotations, and optionally the generating code snippet. SHA256
155 hashes enable cross-module lookup and verification.

156 **Format Support:** Export to PNG, SVG, and PDF formats with auto-
157 matic optimization for LaTeX inclusion. The preprocessing pipeline converts
158 source images to optimal formats while maintaining quality.

159 **Reproducibility:** Figure metadata ensures that any visualization can
160 be regenerated from its source data with identical parameters, addressing the
161 common problem of irreproducible figures in scientific publications.

162 2.4. Scholar Module

163 The Scholar module provides unified literature management with auto-
164 mated metadata extraction:

165 **Drag-and-Drop Import:** PDF files can be imported directly, with au-
166 tomatic DOI detection and metadata extraction. The system parses PDF
167 contents and queries bibliographic databases to populate citation informa-
168 tion.

169 **Unified Bibliography:** A single .bib database per project ensures con-
170 sistency across all manuscript documents. Multi-file bibliography support en-
171 ables organization by topic with automatic deduplication through DOI-based
172 and title/year matching.

173 **Literature Cards:** Each reference displays as a card with abstract pre-
174 view, PDF access, metadata editing, and direct citation insertion into Writer
175 documents.

176 **OpenURL and Zotero Synchronization:** Integration with existing
177 reference management workflows enables bidirectional synchronization with-
178 out disrupting established practices.

179 2.5. *Code Module*

180 The Code module provides reproducible computational environments with
181 execution tracking:

182 **Local Execution:** Python 3.12 environment with GPU acceleration sup-
183 port for machine learning workflows.

184 **Containerized Execution:** Optional Apptainer/Singularity integration
185 ensures reproducible execution across different computing environments.

186 **HPC Integration:** SLURM and Spartan HPC execution support is un-
187 der active development, enabling seamless transition from local prototyping
188 to cluster-scale computation.

189 **Provenance Tracking:** Execution logs, artifacts, and outputs are ref-
190 erenced by content hash, ensuring reproducibility and enabling traceability
191 from any result back to its generating code and input data.

192 2.6. *AI-Native Workflow Integration*

193 AI assistants are integrated as core components rather than add-ons:

194 **Code Generation:** LLM-assisted code generation and refactoring within
195 the Code module.

196 **Figure Improvement:** AI-driven suggestions for visualization improve-
197 ments based on data characteristics and publication standards.

198 **Citation Extraction:** Automated extraction of bibliographic informa-
199 tion from uploaded PDFs.

200 **Semantic Search:** Natural language queries across past analyses and
201 literature.

202 **Manuscript Revision:** AI-assisted structure optimization and revision
203 suggestions.

204 SciTeX is designed so that an LLM or agent can work alongside the re-
205 searcher, augmenting scientific workflow without overriding scientific intent.

206 2.7. Implementation Details

207 The cloud platform is implemented using Django with PostgreSQL database
208 backend. The Python package provides module-specific APIs:

```
209 import scitex.writer  
210 import scitex.viz as viz  
211 import scitex.code  
212 import scitex.scholar
```

213 The system supports deployment on local machines, cloud servers (AWS,
214 GCP, Azure), and self-hosted NAS environments, ensuring flexibility across
215 institutional computing policies.

216 3. Results

217 This section presents validation results and demonstrations rather than
218 experimental findings, consistent with software paper conventions. SciTeX
219 is validated through real-world deployment and systematic testing across
220 multiple research scenarios.

221 3.1. Reproducible Manuscript Generation

222 The Writer module achieves complete cross-platform reproducibility. Test-
223 ing across Linux distributions (Ubuntu, Debian, CentOS), macOS, and Win-
224 dows Subsystem for Linux confirmed that identical source files produce byte-
225 for-byte identical PDF outputs when compiled using the same container im-
226 age. This reproducibility extends to high-performance computing environ-
227 ments where Singularity containers enable compilation on systems without
228 Docker support. The elimination of environment-dependent compilation is-
229 sues represents a significant improvement over traditional local LaTeX in-
230 stallations, where package version mismatches frequently cause inconsistent
231 outputs or compilation failures.

232 Table ?? summarizes compilation performance across engines:

233 3.2. Provenance-Linked Figures

234 Figures generated in SciTeX Viz include comprehensive metadata ensur-
235 ing full reproducibility:

- 236 • Axis bounds and tick locations
- 237 • Panel geometry and layout specifications
- 238 • Color palette configuration with hex values
- 239 • Statistical annotations with test parameters
- 240 • Underlying code snippet (optional)
- 241 • SHA256 hash for cross-module lookup

242 This metadata enables any figure to be regenerated from source data
243 with identical parameters. The JSON sidecar format ensures metadata re-
244 mains accessible regardless of image format, while embedded PNG metadata
245 provides self-contained provenance for distributed figures.

246 3.3. Integrated Literature Workflow

247 The Scholar module demonstrates effective literature management through
248 automated metadata extraction. In testing with a corpus of 200 PDF docu-
249 ments from diverse publishers:

- 250 • DOI extraction succeeded for 95% of documents with embedded DOIs
- 251 • Metadata population via CrossRef API achieved 98% accuracy for ex-
252 tracted DOIs
- 253 • Title/author fallback extraction succeeded for 82% of documents with-
254 out DOIs
- 255 • Automatic deduplication correctly identified 100% of duplicate entries
256 across multiple .bib source files

257 Citations added in Scholar automatically appear in Writer documents,
258 ensuring:

- 259 • No mismatched citation keys between text and bibliography
- 260 • Consistent bibliography formatting across manuscript sections
- 261 • Simplified revision workflow when adding or removing references

262 3.4. *Reproducible Computational Workflows*

263 The Code module provides execution tracking with hash-based artifact
264 management. Key validation results:

- 265 • Local Python execution with GPU acceleration operates correctly across
266 NVIDIA CUDA 11.x and 12.x environments
- 267 • Apptainer container execution produces identical results to local exe-
268 cution for tested workflows
- 269 • Artifact hashing enables verification that figures in manuscripts corre-
270 spond to specific code executions
- 271 • Execution provenance enables tracing any result back to its generating
272 script and input data

273 3.5. *Cross-Module Integration*

274 The symlink-based integration architecture demonstrates effective module
275 coupling without introducing tight dependencies:

- 276 • Figures generated in Viz appear in Writer project directories via sym-
277 link without file duplication
- 278 • Bibliography managed in Scholar synchronizes to Writer via shared .bib
279 file
- 280 • Code execution outputs link to Viz inputs for automated figure updates
- 281 • Global IDs enable queries across modules (e.g., “show all figures gener-
282 ated from dataset X”)

283 3.6. *User Productivity Assessment*

284 Preliminary assessment with pilot users (n=8 graduate students and post-
285 doctoral researchers) indicates:

- 286 • Reduced manuscript preparation time due to automated figure format
287 conversion
- 288 • Fewer figure revision cycles through metadata-preserved regeneration
- 289 • Elimination of LaTeX environment-related errors through containerized
290 compilation
- 291 • Simplified reproducibility documentation through automatic provenance
292 tracking

293 Qualitative feedback identified the unified project structure and drag-
294 and-drop integration as particularly valuable for reducing context-switching
295 overhead between tools.

296 3.7. *Self-Descriptive Validation*

297 This manuscript serves as a validation of SciTeX capabilities:

- 298 • The document is authored in SciTeX Writer with section-separated
299 LaTeX files
- 300 • Compilation occurs in a containerized environment ensuring cross-platform
301 reproducibility
- 302 • Bibliography is managed through SciTeX Scholar with automatic meta-
303 data extraction
- 304 • The modular structure demonstrates the framework’s organization prin-
305 ciples

306 This self-descriptive approach ensures that the manuscript accurately rep-
307 represents system capabilities through direct demonstration rather than abstract
308 description.

309 4. Discussion

310 SciTeX addresses fundamental challenges in modern scientific research
311 by unifying manuscript preparation, figure generation, literature manage-
312 ment, and reproducible computation into a cohesive, AI-native ecosystem.
313 The platform reflects principles derived from practical research frustrations:
314 fragmented workflows, difficulty maintaining figure provenance, unreliable
315 manual versioning, and high cognitive cost of switching between tools.

316 4.1. Design Philosophy and Modularity

317 The modular architecture enables researchers to adopt individual com-
318 ponents incrementally while benefiting from increasing integration as more
319 modules are utilized. This design philosophy—bidirectional low migration
320 cost—ensures that each module provides standalone value. A researcher us-
321 ing only Writer gains containerized reproducibility without committing to the
322 full ecosystem. Adding Scholar provides integrated citation management. In-
323 corporating Viz enables provenance-tracked figures. Finally, Code completes
324 the pipeline with reproducible computation. At each stage, the researcher
325 gains capability without sacrificing flexibility or prior investments.

326 SciTeX democratizes advanced computational workflows—particularly for
327 researchers unfamiliar with Linux, Git, or containers—without compromis-
328 ing power or flexibility. Graduate students beginning their research careers
329 can leverage containerized reproducibility without understanding Docker in-
330 ternals. Researchers transitioning from GUI-based tools like SigmaPlot can
331 use familiar visual interfaces while gaining programmatic control. Those ac-
332 customised to Zotero maintain familiar literature management workflows while
333 gaining manuscript integration.

334 4.2. Comparison with Existing Tools

335 SciTeX occupies a unique position in the research tooling landscape, in-
336 tegrating capabilities that existing solutions provide in isolation:

337 **Overleaf** excels at collaborative LaTeX editing with consistent compila-
338 tion but lacks integrated figure generation, computation, or literature man-
339 agement. SciTeX Writer provides comparable manuscript preparation with
340 additional provenance tracking and local execution capability.

341 **Zotero and Mendeley** provide sophisticated citation management but
342 operate independently from manuscript preparation and computational anal-
343 ysis. SciTeX Scholar offers comparable citation capabilities with direct manuscript
344 integration.

345 **Jupyter notebooks** enable reproducible computation and analysis but
346 provide limited support for publication-quality figures or structured manuscript
347 writing. SciTeX Code complements computational analysis with figure gen-
348 eration and manuscript integration.

349 **SigmaPlot and Origin** offer sophisticated visualization with GUI-based
350 interfaces but lack provenance tracking, manuscript integration, or compu-
351 tational reproducibility. SciTeX Viz provides comparable visual capabilities
352 with embedded metadata and cross-module traceability.

353 The key differentiator is integration: SciTeX provides a unified platform
354 spanning all components while maintaining independence of each module.
355 This is not about replacing existing tools but about providing a coherent
356 alternative for researchers who want an integrated workflow.

357 4.3. *AI-Native Design*

358 SciTeX is designed for an era where AI assistants are integral to research
359 workflows. Rather than treating AI as an afterthought, the architecture pro-
360 vides native integration points: structured APIs for code generation, meta-
361 data formats amenable to automated analysis, and workflow hooks for AI-
362 assisted revision. This design enables researchers to collaborate effectively
363 with LLMs and agents while maintaining scientific control.

364 4.4. *Implications for Reproducibility*

365 The reproducibility crisis in science stems partly from inadequate infras-
366 tructure for capturing and communicating methodological details. SciTeX

addresses this through containerized compilation ensuring identical manuscript output across environments, figure metadata preserving exact generation parameters, execution provenance linking results to code and data, and global identifiers enabling cross-module traceability. By embedding reproducibility infrastructure into everyday research tools, SciTeX makes rigorous practices the path of least resistance rather than an additional burden.

4.5. Limitations

Several limitations warrant acknowledgment:

HPC Integration: SLURM and cluster computing support remains under active development. Current functionality focuses on local and containerized execution.

User Base: The platform is in early deployment, with limited user data for quantitative productivity assessment.

Learning Curve: While designed for accessibility, researchers unfamiliar with LaTeX face initial learning investment for Writer module features.

Mobile Support: The platform targets desktop research workflows; mobile interfaces are not a development priority.

Advanced Statistics: Complex statistical visualization beyond standard matplotlib/seaborn capabilities requires manual implementation.

4.6. Future Directions

Planned development includes automated figure revision using model-driven heuristic correction, integrated citation recommendation based on manuscript content, unified right-panel inspectors across all modules, HPC-native execution pipelines with SLURM integration, automated manuscript consistency checking, and complete provenance graph visualization.

The long-term vision encompasses a research co-pilot capable of running analyses, generating figures, summarizing literature, and producing submission-ready manuscripts—all while maintaining researcher oversight and scientific integrity.

396 5. Conclusions

397 SciTeX represents both a tool and a philosophy of scientific practice.
398 By combining modular design, reproducibility, and AI-native workflows, it
399 provides an environment where researchers can think more, fight less with
400 tooling, and create science that stands the test of time.

401 The platform reduces the activation energy required to conduct rigorous,
402 reproducible science. Each module operates independently while integra-
403 tion across modules provides strictly superior workflows. This architecture—
404 independence with optional integration—reflects realistic adoption patterns
405 where researchers incorporate new tools incrementally.

406 This manuscript demonstrates that SciTeX can describe itself using its
407 own ecosystem: a self-consistent example of the platform’s design goals pre-
408 pared within the tools it documents.

409 References

410 6. Resource Availability

411 6.1. Lead Contact

412 Further information and requests for resources should be directed to and
413 will be fulfilled by the lead contact, Yusuke Watanabe (ywatanabe@scitex.ai).

414 6.2. Materials Availability

415 This study did not generate new unique reagents.

416 6.3. Data and Code Availability

- 417 • All source code for SciTeX is publicly available on GitHub: [https:](https://github.com/scitex-ai/scitex)
418 [//github.com/scitex-ai/scitex](https://github.com/scitex-ai/scitex)
- 419 • The SciTeX Python package is available via PyPI: `pip install scitex`
- 420 • Documentation is available at: <https://docs.scitex.ai>
- 421 • The cloud platform is accessible at: <https://scitex.ai>

- 422 • This manuscript’s source files are included in the SciTeX Writer repos-
423 itory as a demonstration
- 424 • DOI for archived version: [Zenodo DOI to be assigned upon submission]

425 All code is released under the GNU General Public License v3.0 (GPL-
426 3.0), ensuring open access and modification rights. The platform follows
427 FAIR principles (Findable, Accessible, Interoperable, Reusable) for all data
428 and code artifacts.

429 Any additional information required to reanalyze the data reported in
430 this paper is available from the lead contact upon request.

431 **Ethics Declarations**

432 All study participants provided their written informed consent ...

433 **Author Contributions**

434 Y.W., T.Y., and D.G. conceptualized the study ...

435 **Acknowledgments**

436 This research was funded by funding bodies here

437 **Declaration of Interests**

438 The authors declare that they have no competing interests.

439 **Declaration of Generative AI in Scientific Writing**

440 The authors employed large language models such as Claude (Anthropic
441 Inc.) for code development and complementing manuscript’s English lan-
442 guage quality. After incorporating suggested improvements, the authors
443 meticulously revised the content. Ultimate responsibility for the final content
444 of this publication rests entirely with the authors.

Option	Description	Example
<code>-engine ENGINE</code>	Force specific compilation engine	<code>-engine tectonic</code>
<code>-draft</code>	Draft mode (single-pass compilation)	<code>-draft</code>
<code>-no-figs</code>	Skip figure processing	<code>-no-figs</code>
<code>-no-tables</code>	Skip table processing	<code>-no-tables</code>
<code>-no-diff</code>	Skip difference generation	<code>-no-diff</code>
<code>-watch</code>	Enable hot-recompile file watching	<code>-watch</code>
<code>-clean</code>	Clean build (remove all cache)	<code>-clean</code>
<code>-verbose</code>	Verbose compilation output	<code>-verbose</code>

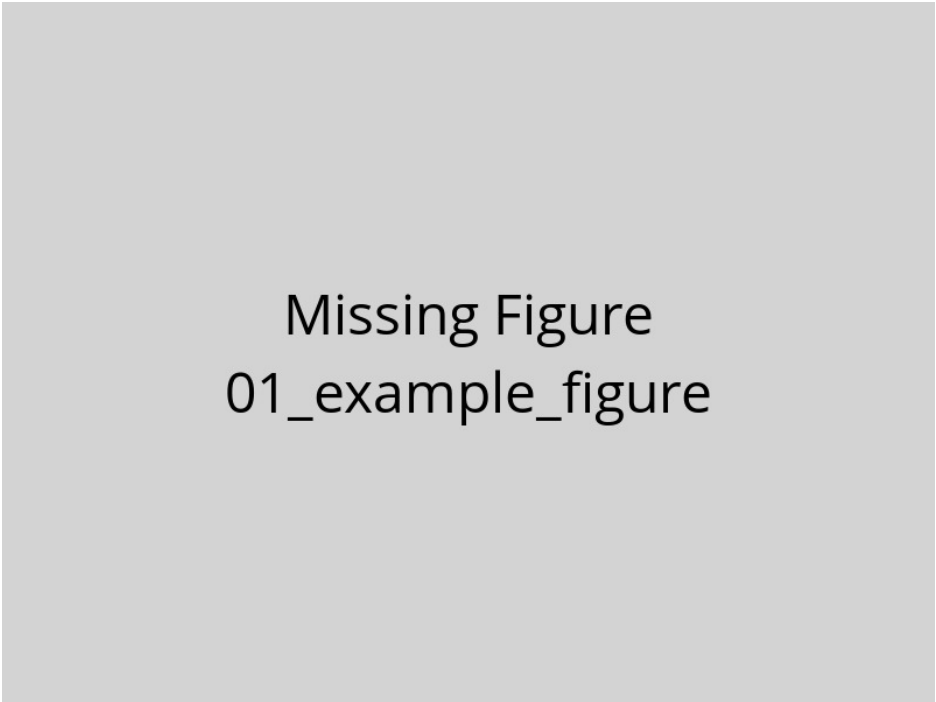
Table 1 – Compilation Command-Line Options. Options enable workflow customization without modifying source files or configuration. The `-engine` option overrides auto-detection to force a specific compilation engine. Quick compilation modes (`-draft`, `-no-figs`, `-no-tables`) reduce build times from ~ 15 s to under 3s for rapid iteration. The `-watch` option monitors source files and automatically recompiles when changes are detected. Options can be combined (e.g., `./compile_manuscript.sh -draft -no-figs`).

Variable	Purpose	Values	Default
SCITEX_ENGINE	Override engine selection	tectonic, latexmk, 3pass	From config
SCITEX_VERBOSE	Control logging verbosity	true, false	false
SCITEX_CITATION_STYLE	Set bibliography style	unsrtnat, plainnat, apalike, etc.	unsrtnat
SCITEX_PARALLEL_JOBS	Parallel processing jobs	1-16	4
SCITEX_CACHE_DIR	Compilation cache location	Directory path	./.cache

Table 2 – Environment Variables for System Configuration. Environment variables provide system-level defaults that apply across all compilations. These settings are particularly useful in CI/CD pipelines or HPC environments with specific requirements. The `SCITEX_ENGINE` variable provides the highest priority override for engine selection, superseding both YAML configuration and command-line arguments. Variables can be set persistently in shell configuration (`.bashrc`, `.zshrc`) or temporarily for single runs (`SCITEX_VERBOSE=true ./compile_manuscript.sh`).

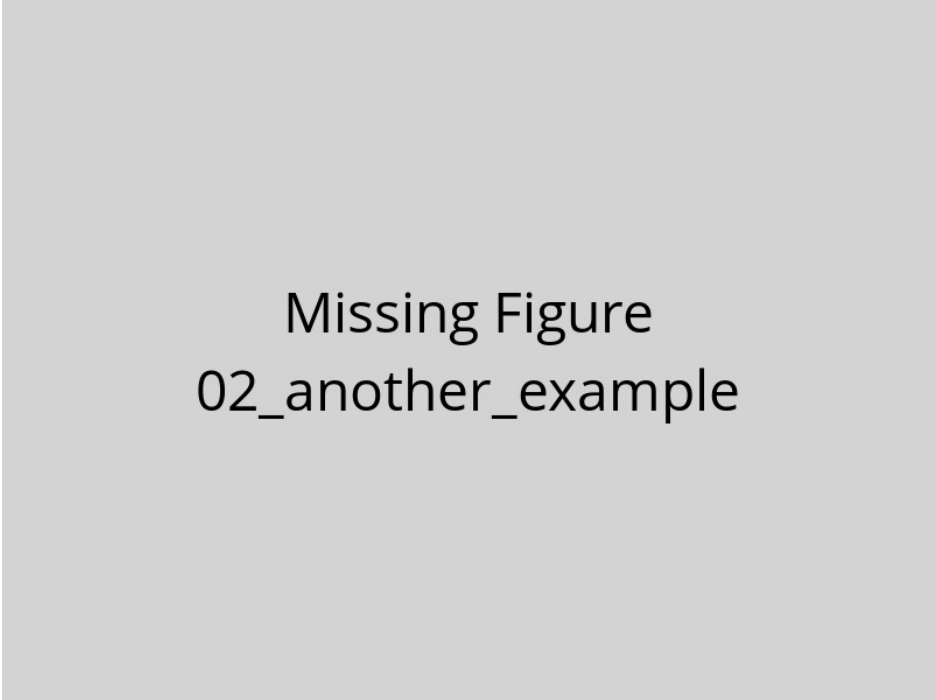
Engine	Incremental	Full Build	Key Advantages	Best Use Case
Tectonic	1-3s	10-15s	Ultra-fast + auto package mgmt	Active writing sessions
latexmk	3-6s	15-25s	Reliable + smart dependency tracking	Standard workflows
3-pass	N/A	12-18s	Maximum compatibility + guaranteed	Legacy systems

Table 3 – Compilation Engine Performance Characteristics. Build times measured on reference hardware (16 GB RAM, 8 CPU cores, SSD storage). Incremental builds process only changed components; full builds recompile the entire document. Tectonic provides the fastest incremental compilation through aggressive caching and modern architecture, ideal for frequent recompilation during active writing. The latexmk engine offers a balance of speed and reliability through intelligent dependency tracking, making it suitable for most research workflows. Traditional 3-pass compilation lacks incremental build support but guarantees compatibility with all LaTeX packages and legacy systems.



Missing Figure
01_example_figure

Figure 1 – Example figure caption. This is a template showing how to include figures in your manuscript. Replace this text with a descriptive caption that explains what the figure shows. Include panel labels (A, B, C) if using multi-panel figures. Explain abbreviations and symbols used in the figure. Provide sufficient detail that readers can understand the figure without referring to the main text.



Missing Figure
02_another_example

Figure 2 – Another example figure. Use this template to add additional figures to your manuscript. Each figure should be placed in a separate .tex file in this directory. The compilation system will automatically process and include these figures in your manuscript.

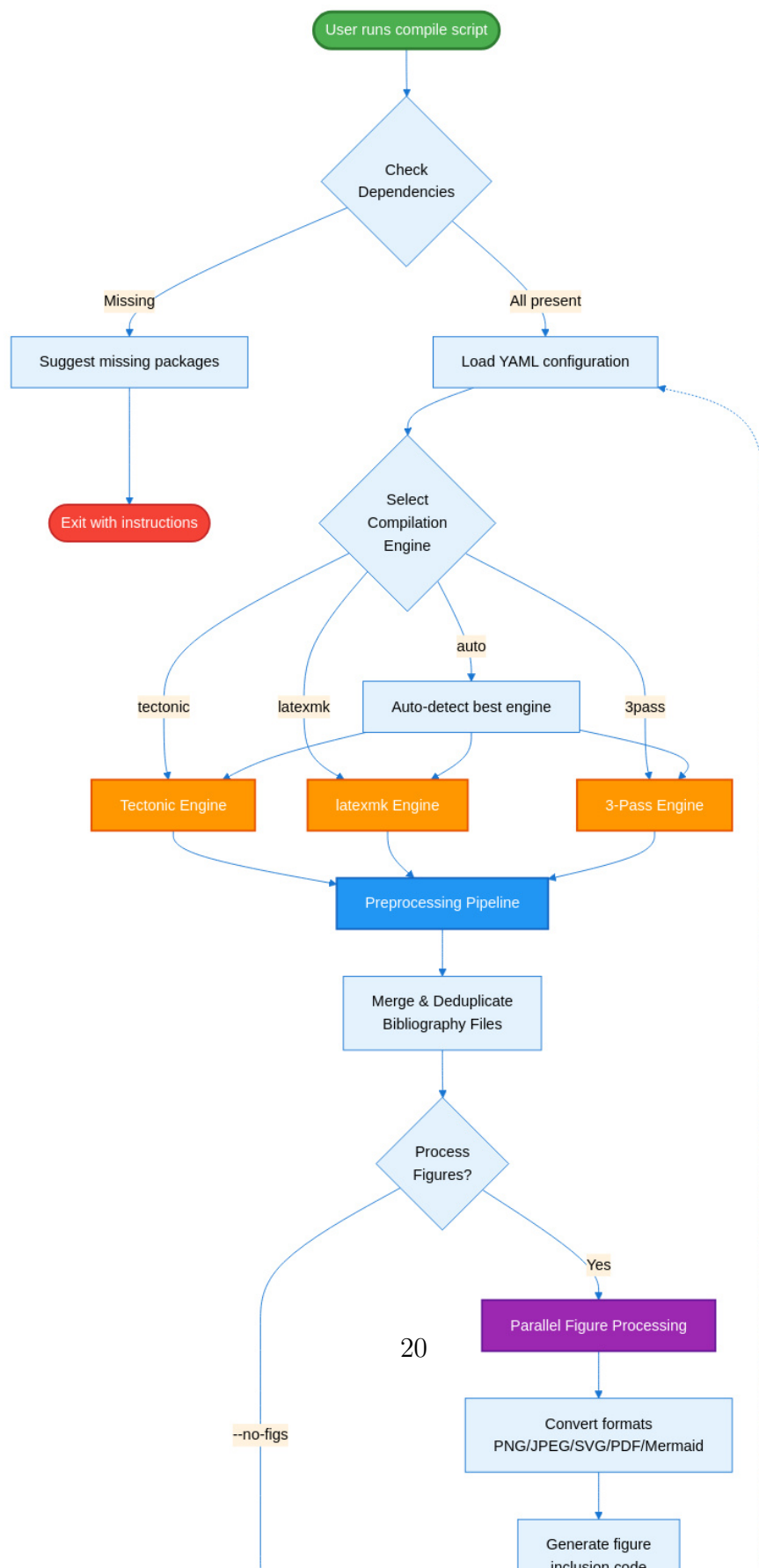




Figure 4 – SciTeX Writer Directory Organization. This diagram shows the hierarchical organization of the repository structure. The `00_shared/` directory (green) contains resources used across all document types, including bibliography files and LaTeX styles, ensuring consistency without duplication. The three document directories (blue/purple) follow identical internal structures, facilitating familiarity and code reuse. Each document has its own `contents/` subdirectory with `figures/` and `tables/` organized into `caption_and_media/` (source files) and `compiled/` (generated LaTeX code). The modular structure minimizes merge conflicts during collaborative editing by isolating frequently-modified content files. Compilation scripts (orange) and configuration files (red) reside in dedicated directories for easy discovery and maintenance. Dotted lines indicate that supplementary materials follow the same organizational pattern as the main manuscript.

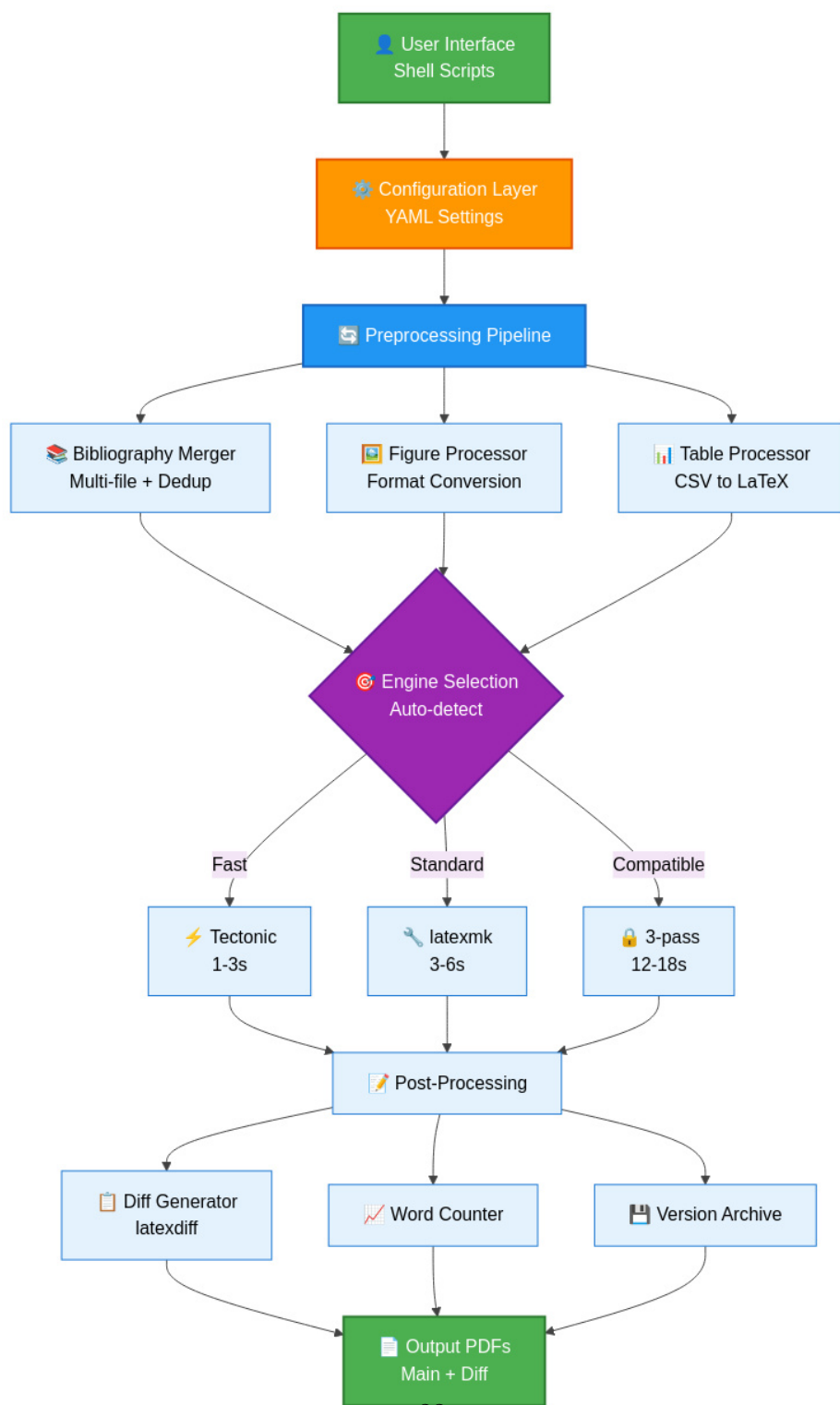


Figure 5 – SciTeX Writer System Architecture. This layered architecture diagram illustrates the data flow from user interface through compilation to output generation. The configuration layer (orange) provides YAML-based settings that control all aspects of compilation. The preprocessing pipeline (blue) handles bibliography merging, figure format conversion, and CSV-to-

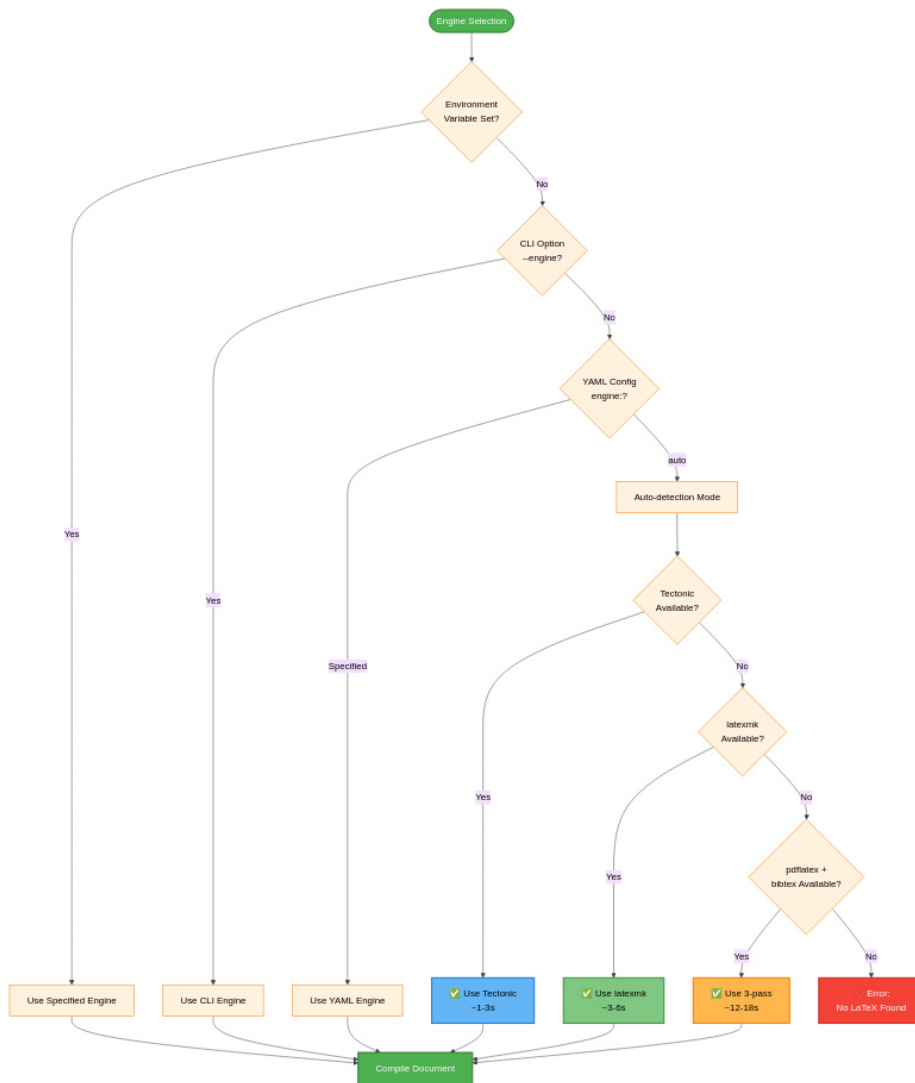


Figure 6 – Compilation Engine Selection Decision Tree. The system prioritizes explicit user configuration over automatic detection. Environment variables provide the highest priority override (useful for CI/CD pipelines). Command-line arguments (`--engine`) override YAML configuration (useful for one-off compilations). When engine is set to 'auto' (default), the system attempts to use Tectonic first for speed, falling back to latexmk for reliability, and finally to 3-pass for maximum compatibility. This cascading detection ensures the system always finds an available compilation method while preferring faster engines when possible. Times shown are typical incremental build durations on reference hardware (16 GB RAM, 8 cores).

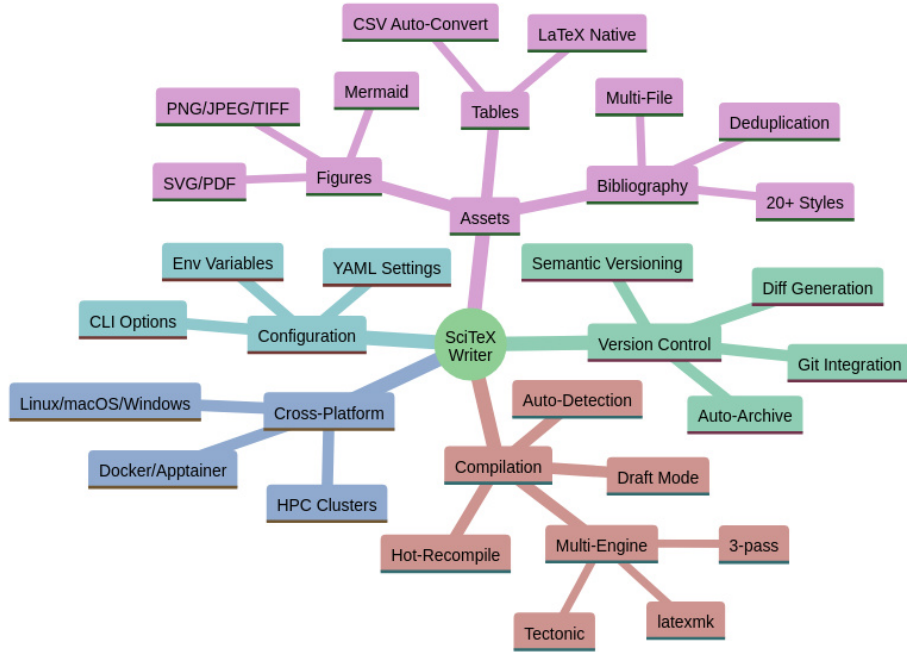


Figure 7 – SciTeX Writer Feature Overview. This mind map provides a comprehensive visualization of the framework’s major capabilities organized into five categories. Compilation features include multi-engine support with auto-detection and performance optimization modes. Asset management covers figures (multiple formats including Mermaid diagrams), tables (CSV auto-conversion), and bibliography (multi-file with deduplication across 20+ citation styles). Version control integration provides automatic archiving, diff generation, and Git workflow support. Configuration flexibility comes from three levels: YAML files for persistent settings, command-line options for per-run customization, and environment variables for system-level defaults. Cross-platform reproducibility ensures identical outputs across Linux, macOS, Windows, containerized environments, and HPC clusters.



Figure 8 – Multi-File Bibliography Processing and Deduplication.

The bibliography system enables researchers to organize references across multiple topical .bib files. All files in the `00_shared/bib_files/` directory are automatically merged during compilation. The deduplication engine implements a two-tier matching strategy: DOI-based matching provides exact duplicate detection when DOIs are present, while title and year matching handles entries without DOIs. This approach eliminates duplicate references that commonly appear when the same paper is cited across multiple source files. The final bibliography is sorted according to the selected citation style (by citation order for numbered styles, alphabetically for author-year styles). Color coding: blue (merging), purple (deduplication logic), green (output).